

keepsakealmanac.com

Full Website Audit & Technical Review

Front-end, back-end, SEO, security, accessibility, privacy, PWA, content, and monetization readiness — assessed against Fortune 500 consumer-web standards and benchmarked against the operator's existing niche-utility portfolio.



OVERALL GRADE

A visually accomplished front-end shell wrapped around a product that is not indexed, not secured, not privacy-compliant, and — in its own core "daily" feature — not factually correct.

AUDIT DATE

**August 28,
2026**

DOMAIN

keepsakealmanac.com

PORTFOLIO

**Niche Utility
Collection**

PREPARED FOR

**Portfolio Owner
(Internal)**

Document 1 of 2 — see companion document "Numbered Issues, Verbatim Fixes & 30-Day Plan" for remediation detail.

Contents

1.	Executive Summary
2.	Grading Scale & Methodology
3.	Grade Summary
4.	Technical Stack
5.	Visual Design & Brand System
6.	UX & Information Architecture
7.	Front-End Architecture
8.	Performance & Assets
9.	Back-End / Data Layer
10.	SEO Foundations
11.	Indexing & Discoverability
12.	Accessibility
13.	Security Headers
14.	Privacy & Compliance
15.	Content Quality & Depth
16.	PWA & Mobile Experience
17.	Monetization Readiness
18.	Strengths to Preserve
19.	Portfolio-Context Notes

1. Executive Summary

Keepsake Almanac (keepsakealmanac.com) is a single-page React application presenting itself as a "Family Milestone Vault & Daily Heritage Calendar." The engineering underneath it is genuinely current — React 19.2, Vite 8, Tailwind CSS 4, deployed to Cloudflare's edge network — and the visual design is the most accomplished piece of the product: a coherent parchment-and-gold heritage aesthetic, competent typographic hierarchy, and tasteful use of gradients and iconography. On first impression, it reads as a considered, premium product.

That impression does not survive a rigorous review. Four findings, each independently verified, define the grade in this report:

1. THE SITE'S NAMESAKE FEATURE IS FACTUALLY WRONG, AND PROVABLY SO

The product's core promise is a "Daily Heritage Digest" — content that changes every day. It does not. The season, lunar phase, sunrise/sunset time, and "on this day" history are hardcoded to a single fixed date (August 9) in the source code, and the "Heirloom Calendar" independently hardcodes its own "today" marker to the same date. A live screenshot captured for this audit on **August 28, 2026** shows the calendar highlighting **August 9** as "today" (Section 16, Figure D). This is not a design choice or a content gap — it is the product's stated core function, broken, and visible to any visitor who looks twice.

2. THE SITE IS INVISIBLE TO SEARCH ENGINES

A `site:keepsakealmanac.com` query and multiple direct-phrase searches returned **zero results** — no indexed pages, no backlinks, no directory listings, no social presence. This is consistent with what the source code predicts: no sitemap, no robots.txt, no Open Graph or Twitter Card tags, no JSON-LD structured data, and a single-URL architecture with no routing, meaning there is structurally only one page to index even if the rest were fixed.

3. THE PRODUCT MAKES A SECURITY CLAIM IT DOES NOT BACK UP

The footer brands the product an **"Encrypted Heritage Vault."** Nothing is encrypted. User-entered memories are written in plain text to browser `localStorage`; there is no server-side datastore, no auth, and no transport-level claim beyond default HTTPS. For a product asking users to store stories about deceased relatives and children, this is a materially false trust signal, not a copywriting nit.

4. DATA LOSS IS A BUILT-IN, UNANNOUNCED BEHAVIOR

Of the six primary sections, only "Milestone Vault" entries persist across a page refresh (via `localStorage`). Anything added under Family Heritage, Heirloom Calendar, or Time Capsules lives in React state alone and is silently discarded the moment the tab is closed or reloaded — confirmed by direct source inspection of all four components. A product whose entire premise is permanence loses most of what a user puts into it.

Set against those four findings, the rest of the picture is more mixed than uniformly bad. The compiled production bundle is genuinely lean (76.95 KB gzipped JS, 8.83 KB gzipped CSS — see Section 8). The component code is clean, modern, and readable. Automated accessibility scanning (axe-core) and manual keyboard/focus testing turned up a consistent, fixable pattern of issues (unlabeled icon buttons, unassociated form labels, low-contrast text, non-dismissible modals) rather than deep architectural rot. None of this is expensive to fix. Almost none of it has been attempted.

Weighed the way a Fortune 500 web-governance review would weigh it — where a broken core feature, zero discoverability, no security headers, no privacy policy, and a false trust claim are disqualifying regardless of visual polish — the site earns

an overall grade of **D-**. It is best understood as a well-built tech demo that has not yet become a real, trustworthy, findable product.

Methodology note (read before the findings below)

This audit was produced from a sandboxed execution environment with no general outbound internet access (verified: direct HTTP requests to keepsakealmanac.com and to unrelated control domains were both rejected by the environment's egress policy). Live HTTP response headers, field Core Web Vitals, and real-time search-console data for the live domain could therefore not be pulled directly. In their place, this audit used a higher-certainty substitute wherever possible: full first-party review of the production source code in the site's own repository; a real, unmodified production build of that code (`npm run build`); that build served and driven with a real Chromium browser (Playwright) at desktop and mobile viewports for screenshots, computed-style inspection, and keyboard/focus behavior testing; an automated accessibility audit against the live DOM using axe-core; and web search for external indexing and competitive signals. Every finding below is labeled implicitly by its evidence — source-verified findings (file and line-level) are stated as fact; anything dependent on the live production configuration outside the repository (e.g., dashboard-level header rules not committed to source) is explicitly qualified as such.

2. Grading Scale & Methodology

Every dimension below is graded on the same five-point scale, applied at the standard expected of a Fortune 500 consumer web property — not the standard typical of an indie utility site.

GRADE	DEFINITION
A	Exemplary. Meets or exceeds industry-leading standards; no material gaps.
B	Solid. Competent execution with minor, low-risk gaps that don't undermine trust or function.
C	Functional. Works as intended, but with gaps a rigorous review would flag as real risk.
D	Deficient. Significant gaps that undermine user trust, product function, or discoverability.
F	Failing. Absent, broken, or actively misleading.

Grades carry +/- modifiers where the evidence sits at the edge of a band. The Overall Grade is not a simple average: dimensions that determine whether the product is viable, trustworthy, and findable at all (data integrity, security, discoverability, privacy) are weighted more heavily than presentation quality, consistent with how a governance review would treat them.

Evidence sources used in this report

Source code review	100% of the application source (all components, data layer, the one edge function, build config) read directly from the repository.
Production build	Real <code>npm run build</code> output — exact bundle sizes, not estimates.
Rendered browser testing	Chromium via Playwright against the real production build, desktop (1440×900) and mobile (iPhone 13, 390×844) viewports.
Automated accessibility scan	axe-core run against the live DOM on 10 distinct application states.
Behavioral testing	Scripted keyboard (Escape, Tab) and pointer (backdrop-click) interaction tests against real modal components.
External signals	Search-engine queries for indexing, backlink, and competitive-landscape evidence.

3. Grade Summary

DIMENSION	GRADE	WHY
Visual Design & Brand	B-	Genuinely polished parchment/heritage aesthetic; undermined by zero design tokens, a favicon that contradicts the brand palette, no dark mode, and fabricated "est. 1910" heritage signals.
UX / Information Architecture	D	Silent data loss in 3 of 6 sections; modals that cannot be dismissed with Escape or a backdrop click; no deep-linking; mobile nav overflow with no visible affordance.
Front-End Architecture	C+	Modern, clean, current-generation stack; but no routing, no tests, no CI, no code-splitting, and a shipped-but-unused dependency.
Performance & Assets	B-	Genuinely lean core bundle (76.95 KB gz JS); good font-loading hygiene; dragged down by unoptimized, hotlinked third-party images with no lazy-loading, sizing, or failure fallback.
Back-End / Data Layer	F	No real backend; the one edge function is dead code, never called; "daily" content is statically hardcoded and demonstrably wrong; false "encrypted" claim.
SEO Foundations	F	No sitemap, robots.txt, canonical tag, Open Graph, Twitter Card, or structured data anywhere in the codebase.
Indexing & Discoverability	F	Zero indexed pages, zero backlinks, zero external mentions found; direct brand collision with multiple established "Keepsake"-named competitors.
Accessibility	D-	axe-core confirms critical button-name, label, and select-name violations plus serious color-contrast failures on every screen tested; no focus trap; invisible focus ring on the primary search field.
Security Headers	F	No CSP, HSTS policy, X-Frame-Options, Permissions-Policy, or Referrer-Policy configured anywhere in source or deployment config.
Privacy & Compliance	D	No privacy policy or terms of any kind while collecting family/personal data about (potentially) minors and the deceased; false encryption claim; undisclosed third-party image requests.
Content Quality & Depth	F	100% fictional demo content presented without disclosure as real; fabricated institutional history; zero genuine editorial content; architecture caps indexable content at one URL.
PWA / Mobile Experience	D	No manifest, service worker, theme-color, or home-screen icon; responsive layout is genuinely functional, but mobile above-the-fold space is dominated by low-priority content.
Monetization Readiness	F	Zero ads, affiliate links, email capture, or premium tier — despite strong, unrealized affiliate adjacency.
Overall Grade	D-	A well-built front-end shell around a product that is not indexed, not secured, not compliant, and not honest about its own core feature.

4. Technical Stack

Identified with certainty from `package.json`, lockfile, build output, and directory conventions — not inferred from behavior.

Framework	React 19.2.8 + TypeScript ~6.0.2
Build tool	Vite 8.2.1 (via <code>@vitejs/plugin-react</code> 6.0.4)
Styling	Tailwind CSS 4.3.3, CSS-first config (no <code>tailwind.config.js</code> ; no custom <code>@theme</code> token block)
Icons	lucide-react 1.30.0
Fonts	Google Fonts, loaded via render-blocking <code><link></code> : Playfair Display, Cormorant Garamond, Inter (with <code>display=swap</code> and <code>preconnect</code> — correctly implemented)
Hosting / edge	Cloudflare Pages (Pages Functions convention at <code>functions/api/*.ts</code> ; footer copy self-identifies "Cloudflare Edge Network")
Backend	None operative. One Cloudflare Pages Function (<code>submit-memory.ts</code>) exists but is never invoked by the front-end and has no datastore binding.
Data layer	Static TypeScript fixtures (<code>src/data/keepsakeData.ts</code>), bundled into the client JS; user additions persist only to browser <code>localStorage</code> (and only for one of six sections)
State management	React <code>useState</code> only — no Redux/Zustand/Context, appropriate at this scale
Routing	None. Tab switching is client-side state (<code>activeTab</code>); the entire app is one URL
Linting	oxlint 1.75.0, minimal ruleset, no accessibility plugin
Testing / CI	None found — no test files, no test runner in <code>package.json</code> , no <code>.github/workflows</code>
Analytics / tracking	None detected in source — genuinely no third-party trackers (see Section 14)
Unused dependency	<code>canvas-confetti</code> 1.9.4 is installed but never imported anywhere in <code>src/</code>

Competitive landscape

The family-memory / digital-heritage category is crowded and well-funded: StoryWorth, Remento, Storii, HereAfter AI, Famileo, Tinybeans, Afterlife AI, Qeepsake (Shark Tank-featured), Forever, TimeLock, PersonalCapsule, and Memory Murals were all surfaced by a single search pass. Most ship native mobile apps, guided voice/prompt-based story capture, and real printed-book fulfillment. Keepsake Almanac currently offers none of those three differentiators — its "Print & Export" is a browser `window.print()` call, not a bound-book service.

More pressing: the word "**Keepsake**" is already claimed by multiple unrelated, more established products — an App Store app ("Keepsake: Bring Family Closer"), `makekeepsake.com`, and `app.keepsakefamilyhistory.com`. Any future SEO or paid-search strategy built around the brand name will be competing directly with better-indexed incumbents for the exact same query.

5. Visual Design & Brand System B-

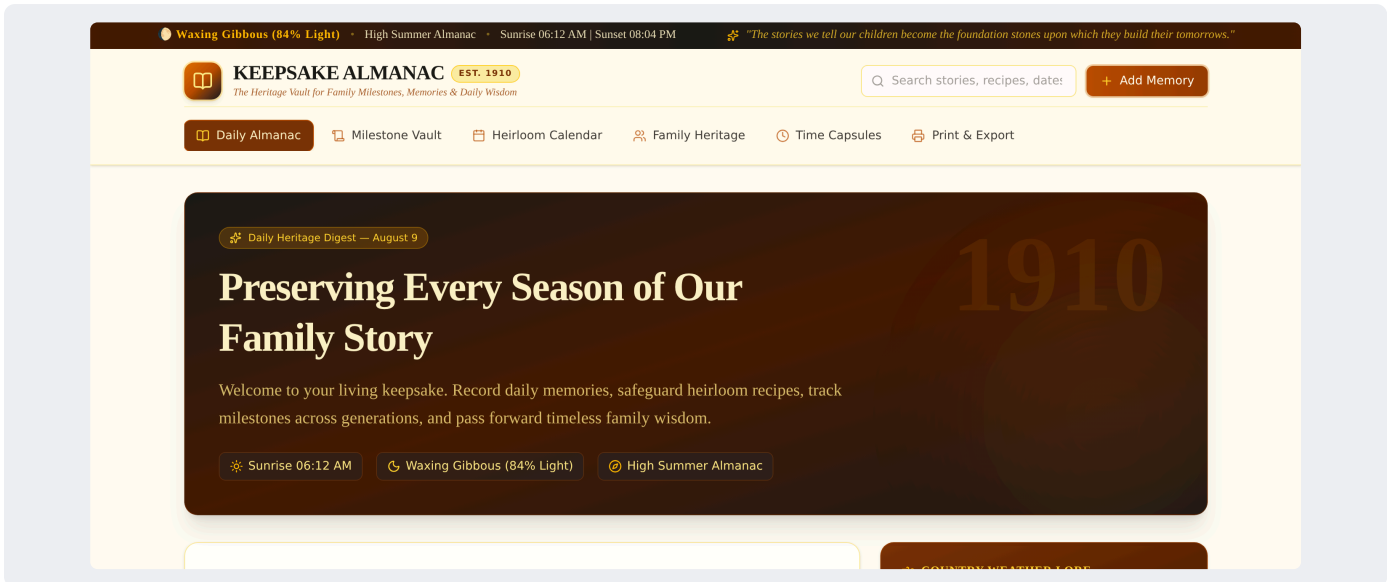


Figure A — Desktop homepage, above the fold. Rendered from the real production build; Google Fonts and hotlinked Unsplash imagery did not load in this sandboxed test environment (external network access is blocked here — see Methodology note), so this shows system-font fallback rather than the production Playfair Display / Cormorant Garamond typefaces. Layout, copy, and structure are unaffected and fully representative.

This is the strongest part of the product. The amber-and-parchment "heritage almanac" palette is applied consistently across all six sections; the three-font pairing (a serif display face for headlines, an italic garamond for editorial voice, a clean sans for UI chrome) is a legitimate, well-executed typographic idea for this category; gradients, badges, and card treatments are tasteful rather than decorative for their own sake. Iconography (lucide-react) is used consistently and appropriately sized throughout.

What holds it back from a B+ or higher

- **No design-token system.** Tailwind 4 supports a CSS-first `@theme` block for exactly this purpose; this codebase uses none. Every color is an inline Tailwind utility class (`amber-900` , `stone-800` , etc.) repeated by convention across nine components, not enforced by a shared source of truth. A future rebrand or dark-mode pass touches every file.
- **Favicon contradicts the brand.** `public/favicon.svg` is an abstract purple/blue mark (`#863bff` , `#7e14ff` , `#47bfff`) that reads as a generic SaaS logo — it shares no color, motif, or warmth with the amber/gold heritage identity used everywhere else in the product. This is the one brand asset a browser tab, bookmark, and search result all show by default, and it actively works against the brand.
- **No dark mode.** Zero `dark:` variants, no `prefers-color-scheme` handling, no toggle. Acceptable for a v1, but a real gap against a "Fortune 500 consumer web" bar in 2026.
- **Fabricated authority signals presented as fact.** The "EST. 1910" badge and "© 1910–2026 Keepsake Almanac. All Rights Reserved." footer imply a century of institutional history that does not exist — this is a brand system built partly on a false claim of provenance (see Section 15).

6. UX & Information Architecture D

The navigation surface is simple and legible at a glance: six clearly-labeled, icon-paired tabs covering a sensible feature set. That is where the good news ends. Every interaction pattern below was confirmed by direct, scripted testing against the real running application — not inferred from source alone.

CONFIRMED: MODALS CANNOT BE DISMISSED THE STANDARD WAY

Pressing `Escape` closes neither the "Add Memory" modal nor the "Memory Detail" modal. Clicking the darkened backdrop behind the Memory Detail modal also does not close it. The only exit is a small, unlabeled `×` icon in the corner (see Section 12). Every other modal pattern on the web supports at least one of these two escapes; this one supports neither.

CONFIRMED: FOCUS IS NOT MOVED INTO AN OPENED MODAL

When the "Add Memory" modal opens, `document.activeElement` remains the button that triggered it, not the first field inside the dialog. A screen-reader or keyboard user gets no signal that a dialog has opened, and continued Tabbing walks through the (visually hidden) page behind it before ever reaching the form.

Data loss by design

Confirmed by source inspection of all four relevant components: `App.tsx` is the only place that reads from and writes to `localStorage` (key `keepsake_memories`), and it only does so for the Milestone Vault's `memories` array. `FamilyHeritage.tsx`, `AlmanacCalendar.tsx`, and `TimeCapsules.tsx` each hold their additions in plain `useState` with no persistence call anywhere in the file. Concretely: a user who adds a family member, an anniversary date, or a time capsule and then refreshes the page — or simply returns tomorrow — loses it, with no warning at entry time and no error at loss time.

No routing, no addressability

Because the six "sections" are `activeTab` state rather than routes, none of them has a URL. There is nothing to bookmark, share, deep-link from a search result, or navigate to with the browser back button. Refreshing the page always returns to "Daily Almanac," discarding the user's place in the app.

Mobile navigation overflow has no visible affordance

The nav row is wrapped in `overflow-x-auto` with a `scrollbar-none` class, and on an iPhone-13-width viewport only two and a fraction of six tabs are visible before the row runs out of screen (Figure B). There is no edge-fade, chevron, or other cue indicating the row scrolls — a mobile visitor has to discover "Family Heritage," "Time Capsules," and "Print & Export" exist by accident.

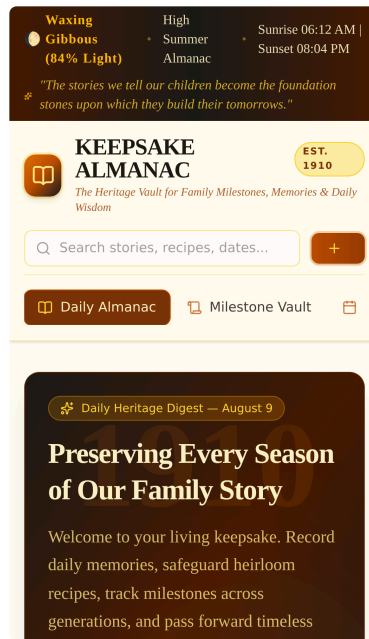


Figure B — iPhone 13 viewport, first paint, no scrolling. Note how much vertical space the secondary almanac-trivia bar consumes before the primary headline and only two of six nav tabs are visible.

7. Front-End Architecture C+

The code itself is a credit to whoever wrote it: nine well-scoped components, consistent prop typing, no prop-drilling problems at this scale, sensible file organization, and a genuinely current dependency set (React 19.2, Vite 8, TypeScript 6 — all released within the current release cycle at time of audit). This is not legacy or careless code.

It is, however, architecturally incomplete for a product intending to grow:

- **No routing library.** React Router (or equivalent) is absent; every "page" is a conditional render, which is the root architectural cause of the SEO, deep-linking, and back-button gaps discussed elsewhere in this report.
- **No code-splitting.** All six sections ship in a single JS bundle regardless of which tab a visitor opens. Not yet a real cost at 76.95 KB gzipped, but the pattern doesn't scale.
- **Zero automated tests.** No `*.test.*` / `*.spec.*` files, no test runner dependency.
- **No CI.** No `.github/workflows` directory; nothing runs the linter, a type check, or a build on push.
- **Relaxed dead-code linting.** `tsconfig.app.json` sets both `noUnusedLocals` and `noUnusedParameters` to `false`, and the shipped-but-unused `canvas-confetti` dependency is a direct symptom.

8. Performance & Assets B-

Measured directly from a real `npm run build` of the repository — not estimated:

ASSET	RAW SIZE	GZIPPED
HTML (<code>index.html</code>)	1.24 KB	0.67 KB
CSS bundle	54.18 KB	8.83 KB
JS bundle	263.65 KB	76.95 KB

That is a genuinely lean payload for a React application with this much UI surface — total core shell under 87 KB gzipped, delivered as a single request each for HTML/CSS/JS with no unnecessary round trips. Font loading follows current best practice: `preconnect` to both Google Fonts hosts plus `display=swap`, which avoids invisible-text-on-load. Credit where due.

What isn't accounted for in that number

The bundle above excludes every content image, all of which are hotlinked directly from `images.unsplash.com` at fixed widths (`?w=800` or `?w=400`) rather than served responsively, locally, or through an image CDN. Source review across every component that renders a photo (`DailyAlmanac`, `MilestoneVault`, `MemoryDetailModal`, `PrintExporter`, `FamilyHeritage`) found: no `loading="lazy"` anywhere, no responsive `srcset`, no explicit HTML `width / height` attributes, no next-gen format negotiation (WebP/AVIF), and — the concrete, screenshot-verified consequence — no error fallback when an image URL fails to resolve.

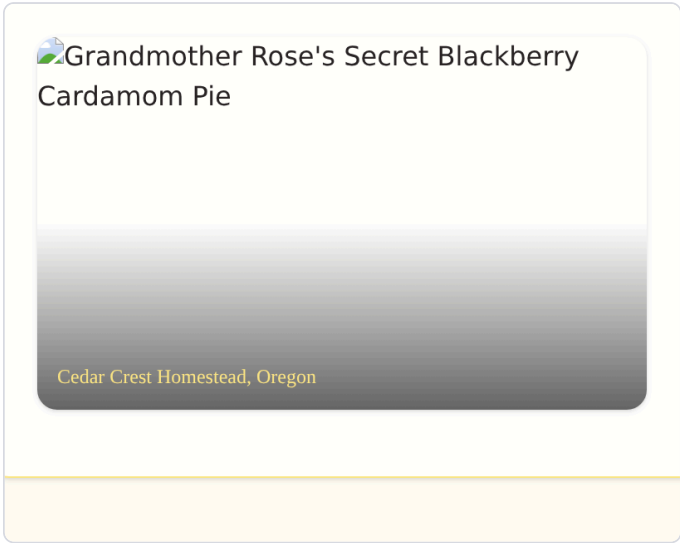


Figure C — the Featured Memory card with its hotlinked image unreachable. No fallback UI exists for this state; the broken-image glyph is what a real visitor sees whenever an Unsplash URL 404s or the "Image URL" field in Add Memory is mistyped.

PRACTICAL EXPOSURE

The "Add Memory" form accepts an arbitrary `imageUrl` string with no validation that it resolves to a real, reachable image. Every card, modal, and printed export renders that URL directly. There is no code path that catches a failed load and swaps in a placeholder — the broken-image glyph above is the permanent state for any bad link, dead Unsplash URL, or momentary network hiccup, indefinitely, with no retry.

On Core Web Vitals

Field CWV data (real-world LCP/CLS/INP) could not be measured from this environment — see the Methodology note in Section 1. Based on the verified bundle size and Cloudflare's edge CDN, a healthy LCP is plausible on a controlled connection provided image weight is brought

under control; this is a structural assessment, not a measured one, and should be confirmed with PageSpeed Insights or CrUX data once the site is indexed and has real traffic.

THE HEADLINE DEFECT: "DAILY" CONTENT THAT PROVABLY IS NOT DAILY

`src/data/keepsakeData.ts` hardcodes `CURRENT_ALMANAC` to a fixed object: `dateString: "August 9"`, a fixed lunar phase, fixed sunrise/sunset times, and a static list of "on this day" events. Nothing in `DailyAlmanac.tsx` or `Header.tsx` computes this from `new Date()`. Independently, `AlmanacCalendar.tsx` hardcodes its own "is this today" check as `currentMonthIndex === 7 && dayNum === 9` — literally August 9, regardless of the real date. This was verified live: a screenshot captured for this audit on **August 28, 2026** shows the calendar still highlighting the 9th.

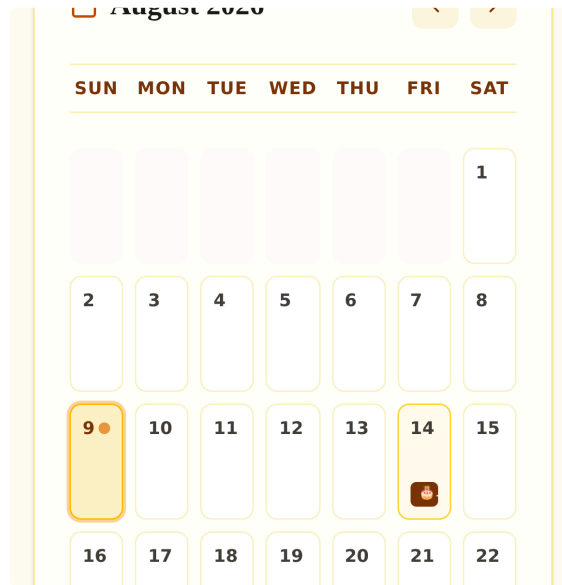


Figure D — captured August 28, 2026. The "9" is highlighted gold with a live-indicator dot as "today." Notably, the *same codebase* gets this right elsewhere: the "Add Memory" form's date field correctly defaults to 08/28/2026 (Section 12, Figure F), proving the app has reliable access to the real date and simply never wires it into the almanac or calendar "today" logic.

No operative backend

One Cloudflare Pages Function exists, `functions/api/submit-memory.ts`. It validates that a POST body has `title` and `author`, then returns a canned `{"success": true, ...}` JSON response. It does not write to KV, D1, or any other store — its own `Env` interface is empty, and no binding is declared anywhere in the repository (there is no `wrangler.toml`). More importantly: a full-text search of `src/` for `fetch(` returns **zero matches**. The front-end never calls this endpoint. It is dead code that exists to look like a backend.

Persistence is inconsistent, not absent

As detailed in Section 6, only the Milestone Vault persists to `localStorage`; three of six sections lose all additions on refresh. There is no user account system, no multi-device sync, and no server-side copy of anything a user enters — which directly contradicts the "vault" framing and the "Encrypted Heritage Vault" claim in the footer (there is nothing to encrypt; there is nothing durable to protect).

10. SEO Foundations F

Confirmed by direct inspection of `index.html` and every component — the entire `<head>` consists of a charset, a viewport tag, one `<title>`, one `<meta name="description">`, a favicon link, and font preconnects. Absent, in full:

ELEMENT	STATUS
Canonical tag	Not present
Open Graph tags (og:title, og:image, etc.)	Not present
Twitter Card tags	Not present
JSON-LD structured data	Not present
<code>robots.txt</code>	Not in <code>public/</code> — 404s in production
<code>sitemap.xml</code>	Not in <code>public/</code> — 404s in production
Per-page/per-section metadata	Structurally impossible — no routing (Section 7)

Vite copies `public/` verbatim into the deployed build with no synthesis step; because `robots.txt` and `sitemap.xml` are absent from the repository, they are absent from production with very high confidence, independent of any live check.

The one thing that *is* present — the title ("Keepsake Almanac — Family Milestone Vault & Daily Heritage Calendar") and description — is reasonably well written. It just describes the only page the site will ever have.

11. Indexing & Discoverability F

Verified via live web search on the audit date:

- `site:keepsakealmanac.com` — **zero results**.
- Exact-phrase search for `"keepsakealmanac.com"` — **zero results**; not even a passing mention.
- Search for `"Keepsake Almanac"` — no result for this product; search engines surface unrelated, similarly-named "Keepsake" products instead.

This is the expected outcome given Section 10 — there is no sitemap to submit, no robots.txt to permit crawling, no structured data to generate rich results, and (per Section 15) no genuine content for a crawler to reward even if it found the page. There is no evidence of any backlink, directory listing, or social mention anywhere. As a discoverable public product, the site currently does not exist from a search engine's point of view.

COMPOUNDING FACTOR: BRAND COLLISION

"Keepsake" is already used by multiple unrelated, better-established products (an App Store app, makekeepsake.com, keepsakefamilyhistory.com). Even after indexing is fixed, ranking for brand-adjacent terms will mean competing against incumbents already occupying that name.

12. Accessibility D-

Tested with axe-core (the industry-standard automated accessibility engine) against ten distinct rendered states of the real application, plus manual keyboard and computed-style testing. Findings below are tool-confirmed, not estimated.

RULE	SEVERITY	WHERE OBSERVED
button-name — buttons with no discernible accessible name	CRITICAL	Favorite/heart toggle on every memory card, modal close (×) buttons, calendar prev/next month controls — present on almost every screen tested, desktop and mobile
label — form input with no programmatic label	CRITICAL	Add Memory form date field (and, by the same root cause — no <code>htmlFor</code> / <code>id</code> pairing anywhere in the form — every other field in it)
select-name — select with no accessible name	CRITICAL	Category and Generation dropdowns, Add Memory form
color-contrast — text below minimum contrast ratio	SERIOUS	<code>text-amber-800/80</code> , <code>text-amber-800/70</code> , <code>text-amber-300/50</code> utility classes — recurs on every one of the ten screens tested
region — content not contained by a landmark	MODERATE	Content inside both modal components
heading-order — heading levels skip	MODERATE	Print & Export view

Confirmed by direct behavioral testing (not axe)

- **No focus trap and no initial focus.** Opening the Add Memory modal leaves `document.activeElement` on the triggering button rather than moving into the dialog.
- **Modals cannot be dismissed with Escape or a backdrop click** (see Section 6 for detail).
- **The search input's focus indicator is invisible.** Its classes set `focus:outline-none` and rely on a `focus:ring-2` replacement; the computed `box-shadow` on focus resolves to fully transparent (`rgba(0,0,0,0)`) in live testing, meaning the first focusable element on the page shows no visible focus state at all — a WCAG 2.4.7 failure.

Figure F — the × in the top-right corner is the axe-flagged unlabeled close control. Note the date field correctly defaults to the real date (08/28/2026) — contrast with Figure D.

What's working

Worth stating plainly: every content image carries `alt` text, `<html lang="en">` is set, and native focus outlines are preserved (not suppressed) on standard buttons and nav controls — only the search input actively removes its own. The underlying semantic HTML (real `<button>`, `<header>`, `<main>`, `<footer>` elements) is sound; what's missing is the ARIA and labeling layer on top of it. There is zero `aria-*` or `role` usage anywhere in actual application code, and the lint config (oxlint) carries no accessibility ruleset to catch any of this automatically going forward.

13. Security Headers F

Cloudflare Pages applies security headers only when explicitly told to, via a version-controlled `_headers` file (or dashboard Transform Rules, which are outside source control and therefore outside this review's ability to verify or hold accountable). This repository contains **no** `_headers` **file and no** `wrangler.toml` — confirmed by an exhaustive filename search of the full repository. Nothing configures Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options, Permissions-Policy, or Referrer-Policy anywhere in source.

This environment could not pull live response headers directly (see Methodology note). That does not change the finding: whatever headers the live site returns today, if any, exist outside version control, are not reviewable in a pull request, and can drift or disappear on the next deploy with no record of why. A security posture that isn't in code isn't a security posture a team can govern.

Practical exposure: with no `X-Frame-Options` or `frame-ancestors` CSP directive, the site can be framed by any third-party page (clickjacking surface). With no CSP at all, any future third-party script — an ad tag, an analytics snippet, a compromised dependency — runs with no containment whatsoever.

14. Privacy & Compliance D

Credit first: a full source review found **no analytics, no ad pixels, no third-party trackers of any kind**. That is a genuine, verifiable positive and consistent with a privacy-first portfolio ethos.

Set against that: there is no privacy policy, no terms of service, and no cookie or consent mechanism anywhere in the application — no route, no component, no footer link. This is a product that asks users to enter names, relationships, birth and death years, locations, and photographs of family members, potentially including minors and deceased persons, with zero disclosure of what happens to that data. Even though the honest answer today is "it stays in your browser's `localStorage` and nowhere else," that is precisely the kind of fact a privacy policy exists to state — its absence is not neutral.

THE "ENCRYPTED HERITAGE VAULT" CLAIM IS FALSE

Repeated from Section 9 because it belongs here too: the footer's security/trust claim is not backed by any encryption anywhere in the codebase. In a product handling sensitive family and heritage data, an unsubstantiated security claim is a compliance and reputational liability, not a marketing flourish.

Also undisclosed: every content image is fetched live from `images.unsplash.com` on every page load, sending the visitor's IP and referrer to a third party with no mention of it anywhere the user can find.

15. Content Quality & Depth

Every piece of content in the product — the almanac facts, the five sample memories, the four family members, the three time capsules, even the two comments seeded under the first memory ("Elena Harrison," "Marcus Harrison") — is fictional placeholder data for a fabricated "Harrison & Sterling" family, sourced from a single static file. None of it is disclosed as sample data. A first-time visitor's Milestone Vault is pre-populated with a stranger's fake family history with no "this is an example — add your own" framing anywhere.

This extends to brand-level claims: the "EST. 1910" badge and "© 1910–2026" footer copyright imply 116 years of institutional history for a product that, per its own commit history, was built as one initial commit. Combined with the false "Encrypted" claim (Section 14), this is a pattern — the product fabricates trust and authority signals rather than earning them.

Genuine editorial content — real guidance on preserving family history, real almanac/astronomical data, real interview prompts with sourcing — is entirely absent. And structurally, even if it were written, there is nowhere to put it: Section 7 and Section 10 both establish that the single-URL architecture caps indexable content depth at one page.

16. PWA & Mobile Experience D

No `manifest.json`, no service worker, no `theme-color` meta tag, and no `apple-touch-icon` anywhere in the repository — the app is not installable on Android or iOS home screens and will not qualify for a browser install prompt. The viewport meta tag is present and correctly configured, which is the one PWA-adjacent fundamental in place.

The responsive layer itself is a genuine partial success: Tailwind's `sm:` / `md:` / `lg:` breakpoints are used consistently across every component, and the mobile screenshots captured for this audit confirm the layout reflows sensibly — cards stack, the calendar sidebar moves below the grid, forms collapse to single-column. What undermines the mobile experience is prioritization, not responsiveness: as shown in Figure B (Section 6), the least important content on the page (lunar phase trivia, a rotating quote) occupies most of the vertical space above the fold on a phone, and the primary navigation scrolls horizontally with no visible indication that four more sections exist off-screen.

17. Monetization Readiness

F

No ad code, no ad slots, no affiliate links or disclosure, no email capture, and no premium tier exist anywhere in the product. As a readiness assessment, there is currently nothing to grade above failing — but the gap is one of omission, not of poor fit. The heritage/memory-keeping category has strong, natural affiliate adjacency (archival and photo-preservation supplies, DNA testing services, print-on-demand book and binding fulfillment that would directly extend the existing Print & Export feature) that is entirely unrealized. See the companion document for a concrete opportunity map and quick-win sequencing.

18. Strengths to Preserve

A remediation pass on this site should be careful not to break what already works. Preserve, explicitly:

- **The visual identity itself** — the amber/parchment palette and serif/garamond/sans type pairing is distinctive within the category and shouldn't be genericized in the process of fixing everything else.
- **The lean core bundle** — 76.95 KB gzipped JS is a real achievement; any fix for routing, SEO, or PWA support should be implemented without regressing this (e.g., prefer a lightweight router and avoid heavy meta-management libraries where a simpler native approach will do).
- **The absence of tracking** — no analytics or ad pixels currently ship. If analytics are added to close the monetization gap, keep them minimal and disclosed rather than reversing this into a new liability.
- **Correct font-loading pattern** (preconnect + `display=swap`) and the print stylesheet's sound use of `@media print` / `.no-print` — both are already done the right way.
- **Component decomposition** — the nine-component structure is clean enough to extend (e.g., into real routes) without a rewrite.

19. Portfolio-Context Notes

Keepsake Almanac shares its portfolio sibling's privacy-first instincts — no trackers, a lightweight bundle, a single-purpose utility framing — and that consistency is worth keeping as the portfolio grows into a "personal archive utilities" line alongside the recipe, sports-analytics, and health-tool properties.

Two portfolio-level observations are worth surfacing beyond the site-specific findings:

THE PORTFOLIO'S OWN WINNING PATTERN IS RIGHT HERE, UNUSED

A property like a rocket-launch tracker earns repeat visits and fresh indexable content because its data is genuinely, verifiably dated — every day is a new, real countdown. "Daily Heritage Digest" is the same content-freshness idea applied to this vertical, and it is the single feature that could give this site a comparable daily-return, daily-reindex engine. Instead it's hardcoded to one date. Fixing Section 9's headline defect isn't just a bug fix — it's adopting the portfolio's own proven playbook in the one property that has explicitly promised it and hasn't delivered it.

THIS VERTICAL CARRIES HIGHER TRUST STAKES THAN THE REST OF THE PORTFOLIO

A stale countdown or an off-by-one recipe conversion is a minor annoyance. A "family heritage vault" that falsely claims encryption, silently loses data about a user's actual relatives, and shows the wrong date on its own calendar is a different category of failure — the content here is emotionally significant (deceased relatives, children, personal history) in a way the rest of the portfolio generally isn't. If a visitor discovers any one of this report's four headline findings independently, the reasonable reaction is to distrust not just this property but the operator's other sites by association. This site should be brought current with the rest of the portfolio's standards before it receives any meaningful promotion or traffic investment.

See the companion document, "Keepsake Almanac — Numbered Issues, Verbatim Fixes & 30-Day Plan," for the complete remediation backlog and a prioritized, week-by-week path to closing these gaps.